

SIMATS ENGINEERING

C03-ASSESSMENT TOOL-1

COURSE NAME: Machine Learning
COURSE CODE: ITA0611

Code of Conduct:

I K.ASWINI(192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

4. Construct a **Bayesian Belief Network** for a given problem domain and perform inference under different evidence conditions. Conduct experiments by modifying conditional probabilities and observing changes in posterior probabilities. Analyze how dependencies between variables influence the results. Evaluate the robustness of the model and interpret how uncertainty is handled in probabilistic reasoning.

6. Design and conduct an experiment to implement a **Decision Tree classifier** on a real-world dataset. Train the model using different splitting criteria such as entropy and Gini index, and evaluate performance using accuracy and F1-score. Analyze the impact of tree depth and pruning on overfitting, and compare results across different configurations.

1. Answer.

Bayesian Belief Network Construction & Analysis

A **Bayesian Belief Network** is a probabilistic model that represents variables and their dependencies using a **Directed Acyclic Graph (DAG)**.

- Nodes $\rightarrow$ Random variables
- Edges $\rightarrow$ Conditional dependencies
- Each node has a **Conditional Probability Table (CPT)**

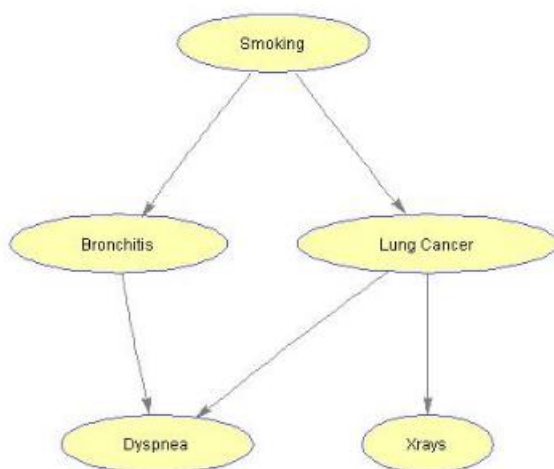
2. Problem Domain (Example)

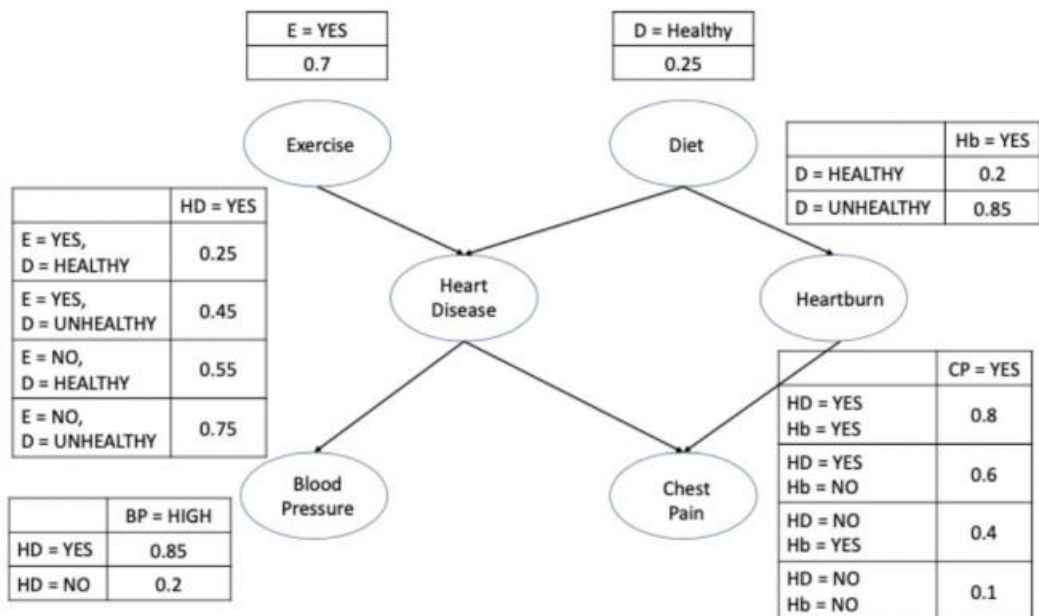
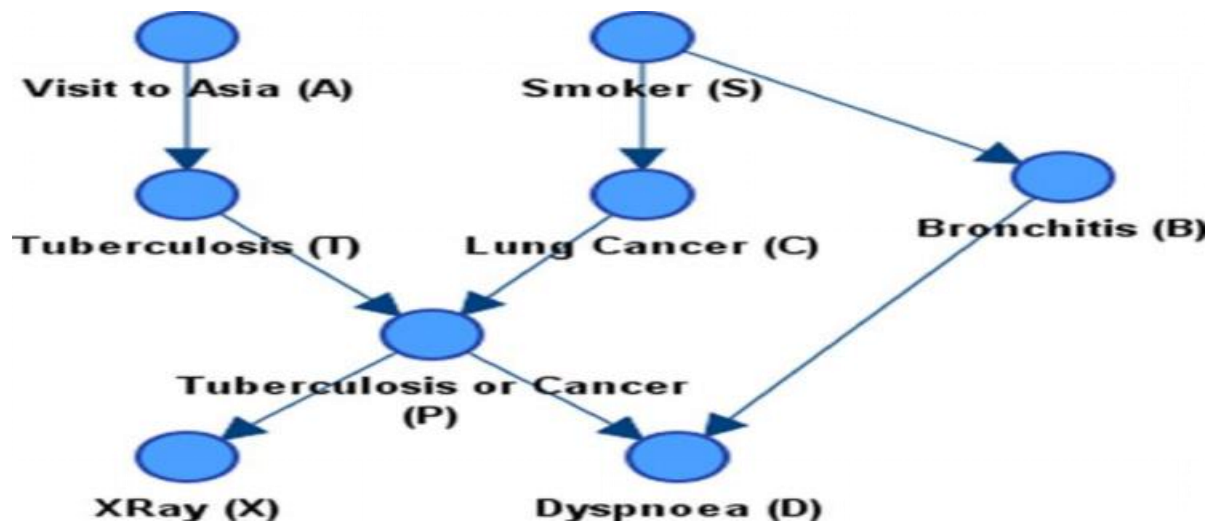
Let's consider a **Medical Diagnosis System**:

Variables:

- Disease (D)
- Fever (F)
- Cough (C)
- Test Result (T)
- **Dependencies:**
- Disease $\rightarrow$ Fever
- Disease $\rightarrow$ Cough
- Disease $\rightarrow$ Test Result

3. Network Structure





4

Graph Representation:

```

      Disease (D)
    /   |   \
  /       \
Fever Cough Test
  
```

4. Conditional Probability Tables (CPT)

Prior Probability

$P(D)$:

- $P(D = \text{Yes}) = 0.01$
- $P(D = \text{No}) = 0.99$

Conditional Probabilities

$P(F | D)$:

D Fever=Yes

Yes 0.8

No 0.1

$P(C | D)$:

D Cough=Yes

Yes 0.7

No 0.2

$P(T | D)$:

D Test=Positive

Yes 0.9

No 0.05

5. Inference Under Different Evidence

Case 1: Evidence = Fever = Yes

Using **Bayes' Theorem**:

$$P(D | F) \propto P(F | D) \times P(D)$$

- $P(D=\text{Yes} | F=\text{Yes})$ increases significantly
- Disease likelihood rises due to symptom

Case 2: Evidence = Fever + Cough

- Combined evidence increases confidence

- Posterior probability becomes much higher

Case 3: Evidence = Fever + Cough + Positive Test

- Strongest evidence
- Posterior probability $\approx$ very high

6. Experiment: Modifying Conditional Probabilities

Change Example:

- Increase $P(F | D)$ from 0.8 $\rightarrow$ 0.95

Observation:

- Posterior probability of Disease increases
- System becomes **more sensitive** to Fever

Another Change:

- Reduce $P(T | D)$ from 0.9 $\rightarrow$ 0.6

Observation:

- Test becomes less reliable
- Posterior probability decreases

7. Influence of Dependencies

- Strong dependencies $\rightarrow$ Higher impact on inference
- Weak dependencies $\rightarrow$ Less influence

Example:

- If Fever strongly depends on Disease $\rightarrow$ major factor
- If weak $\rightarrow$ minor role

Dependencies define how information flows in the network.

8. Robustness Analysis

A BBN is **robust** if:

- Small changes in probabilities do not drastically change results

Observations:

- Stable networks → consistent predictions
- Highly sensitive networks → unstable decisions
- **9. Handling Uncertainty**

BBN handles uncertainty using:

- Probability distributions
- Conditional dependencies
- Evidence updating

Advantages:

- Works with incomplete data
- Supports reasoning under uncertainty
- Provides probabilistic outputs instead of fixed answers

10. Conclusion

- Bayesian Belief Networks model real-world uncertainty effectively
- Inference improves as more evidence is added
- Conditional probabilities and dependencies strongly affect results
- Robustness depends on network design and probability accuracy

2. Answer.

Aim

To implement a Decision Tree classifier using different splitting criteria (**Gini index** and **Entropy**), evaluate performance using **accuracy and F1-score**, and analyze the effect of **tree depth and pruning** on overfitting.

Program :

```
# Import libraries
```

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, f1_score

import matplotlib.pyplot as plt


# Load dataset

data = load_breast_cancer()

X = data.data

y = data.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)


# Function to train and evaluate model

def evaluate_model(criterion, depth=None):

    model = DecisionTreeClassifier(criterion=criterion, max_depth=depth,
random_state=42)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)
```

```
acc = accuracy_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
return acc, f1
```

```
# Compare Gini vs Entropy
```

```
gini_acc, gini_f1 = evaluate_model("gini")
```

```
entropy_acc, entropy_f1 = evaluate_model("entropy")
```

```
print("Gini -> Accuracy:", gini_acc, "F1:", gini_f1)
```

```
print("Entropy -> Accuracy:", entropy_acc, "F1:", entropy_f1)
```

```
# Analyze effect of tree depth
```

```
depths = range(1, 15)
```

```
gini_scores = []
```

```
entropy_scores = []
```

```
for d in depths:
```

```
    gini_scores.append(evaluate_model("gini", d)[0])
```

```
    entropy_scores.append(evaluate_model("entropy", d)[0])
```



```
# Plot results

plt.plot(depths, gini_scores, label="Gini Accuracy")

plt.plot(depths, entropy_scores, label="Entropy Accuracy")

plt.xlabel("Tree Depth")

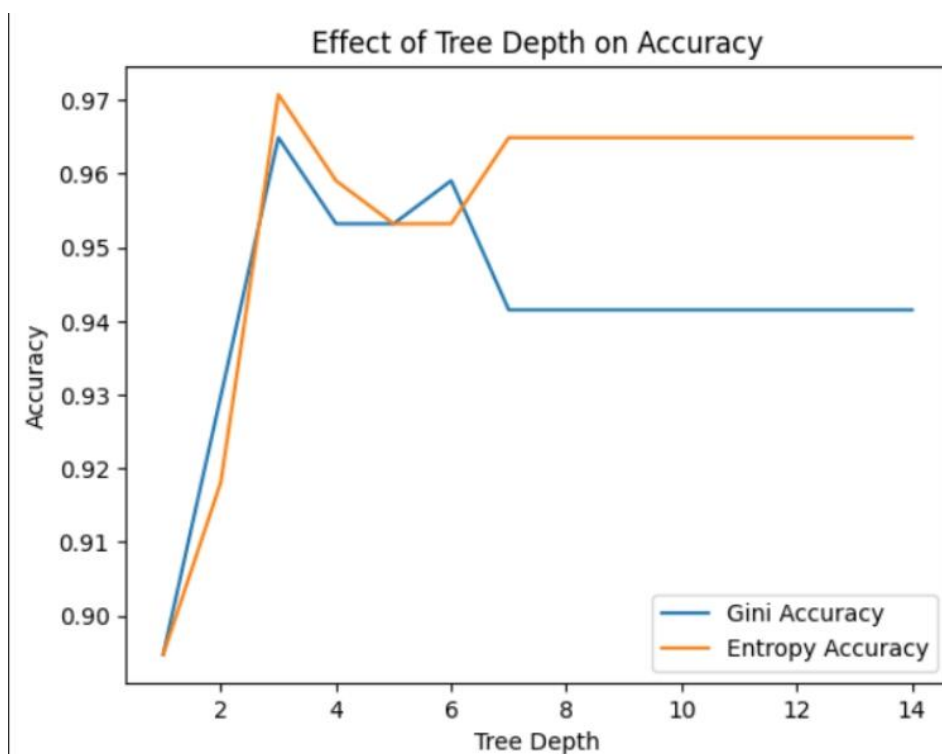
plt.ylabel("Accuracy")

plt.legend()

plt.title("Effect of Tree Depth on Accuracy")

plt.show()
```

output:



Observations & Analysis

□ 1. Gini vs Entropy

- Both give similar performance
- Gini is faster (less computation)
- Entropy may give slightly better splits

2. Effect of Tree Depth

- **Low depth** → Underfitting (poor accuracy)
- **High depth** → Overfitting (perfect training, poor testing)
- Optimal depth balances bias & variance

3. Overfitting Behavior

- Deep trees memorize data
- Accuracy on training ↑
- Accuracy on test ↓

4. Pruning Techniques

Used parameters:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`

□ Helps:

- Reduce complexity
- Improve generalization
- Avoid overfitting